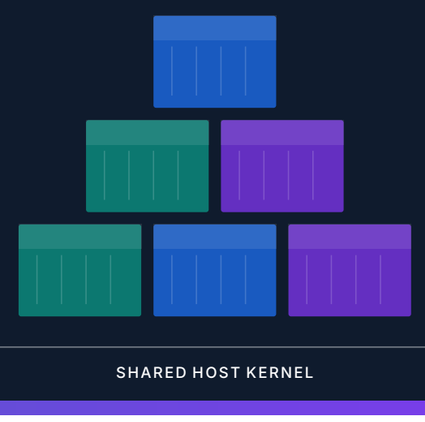
AI ENGINEERING INSIDER

Docker Cheat Sheet

The practitioner's field guide: how containers really work, how to build small and secure images, and how to run them in production, from laptop to GPU node.



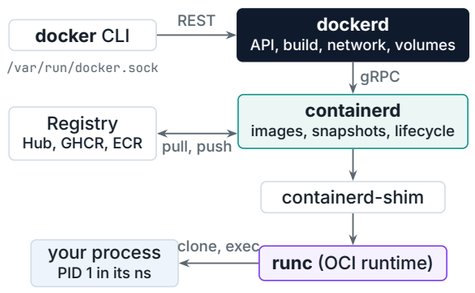
01 The Mental Model

WHAT A CONTAINER ACTUALLY IS

A container is **an ordinary Linux process** with a restricted view of the system. There is no hypervisor and no guest kernel: every container on a host shares that host's kernel. Four kernel features do all the work.

Feature	What it gives the process
Namespaces	Its own view of PIDs, network, mounts, hostname, IPC, users, cgroups
cgroups v2	Hard limits and accounting for memory, CPU, I/O, PIDs
Union FS	Layered, copy-on-write root filesystem (`overlay2`)
Capabilities, seccomp	Root split into fine-grained privileges; syscall filtering

ARCHITECTURE: WHO DOES WHAT

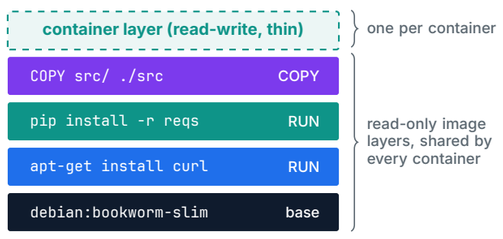


The shim keeps containers alive when `dockerd` restarts (with `live-restore`). On macOS and Windows, Docker Desktop runs all of this inside a small Linux VM, which is why file sharing and networking behave differently there.

> Access to `/var/run/docker.sock` is equivalent to root on the host. Never mount it into a container you do not fully trust.

02 Images and Layers

IMMUTABLE, CONTENT-ADDRESSED, STACKED

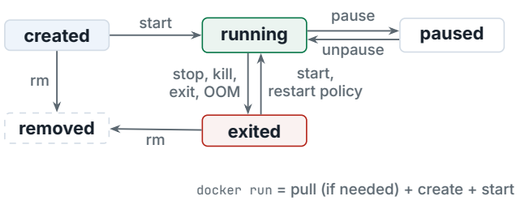


- `RUN`, `COPY` and `ADD` each add a layer; metadata instructions do not.
- Layers are identified by **sha256 digest**; identical layers are stored and pulled once.
- A tag (`:1.4.2`) is a movable pointer. A digest (`@sha256:...`) is immutable.
- Multi-arch images are an **OCI index** pointing at one manifest per platform.
- Writes go to the container layer via copy-on-write; they vanish with `docker rm`.

> Deleting a file in a later layer does not shrink the image. The bytes still live in the earlier layer. Clean up in the *same* `RUN`, or use a multi-stage build.

03 Container Lifecycle

STATES AND THE COMMANDS THAT MOVE THEM



`docker run` = pull (if needed) + create + start

Restart policies: `no` (default), `on-failure[:N]`, `always`, `unless-stopped`. Use `unless-stopped` for long-lived services on a single host so a manual stop survives a daemon restart.

04 CLI Essentials

THE COMMANDS YOU USE EVERY DAY

Command	Purpose
`docker run -d --name api -p 8000:8000 img`	Create and start in the background
`docker run --rm -it img sh`	Throwaway interactive shell
`docker ps -a`	All containers, including exited
`docker logs -f --tail 100 api`	Follow the last 100 log lines
`docker exec -it api sh`	Shell into a running container
`docker stop api / docker rm api`	SIGTERM, then remove
`docker build -t app:1.0 .`	Build from the current context
`docker tag app:1.0 ghcr.io/me/app:1.0`	Add a registry-qualified name
`docker inspect api`	Full JSON config and state
`docker stats`	Live CPU, memory, I/O per container
`docker system df`	Disk used by images, volumes, cache
`docker system prune`	Remove stopped, dangling, unused
`docker init`	Scaffold Dockerfile, compose, ignore

05 Dockerfile Reference

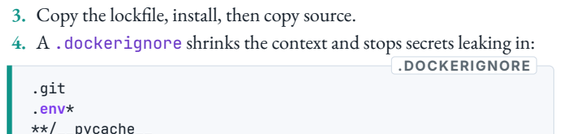
EVERY INSTRUCTION, WITH THE GOTCHA

Instruction	Meaning and gotcha
`FROM img AS name`	Start a stage. Pin by digest in production
`ARG`	Build-time variable. Visible in `docker history`; never for secrets
`ENV`	Runtime variable, persisted into the image
`WORKDIR`	Sets and creates the working directory. Prefer over `cd`
`COPY`	Copy from context or `--from=stage`. Use `--chown`
`ADD`	Like COPY, plus URL fetch and tar auto-extract. Prefer COPY
`RUN`	Execute at build time; one layer per instruction
`USER`	Drop root for every later step and at runtime
`EXPOSE`	Documentation only. Publishing needs `-p`
`VOLUME`	Declares an anonymous volume; later writes there are lost at build
`ENTRYPOINT`	The fixed executable
`CMD`	Default arguments to ENTRYPOINT, easily overridden
`HEALTHCHECK`	Probe that drives `healthy`/`unhealthy` status
`STOPSIGNAL`	Signal sent by `docker stop` (default SIGTERM)
`LABEL`	OCI metadata: source, version, licenses

> Shell form (`CMD python app.py`) runs under `/bin/sh -c`, so `sh` becomes PID 1 and does not forward SIGTERM. Your app is SIGKILLed after 10 seconds. Always use exec form: `CMD ["python", "app.py"]`.

06 A Production Dockerfile

MULTI-STAGE, CACHED, NON-ROOT, HEALTHY



DOCKERFILE

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim AS builder
WORKDIR /app
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
# 1. dependencies first: cached until the lockfile changes
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt

FROM python:3.12-slim AS runtime
ENV PYTHONUNBUFFERED=1 PATH="/opt/venv/bin:$PATH"
RUN useradd --uid 10001 --create-home app
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
# 2. source last: changes often, invalidates little
COPY --chown=app:app src/ ./src/
USER 10001
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request as u; u.urlopen('
     http://127.0.0.1:8000/healthz')"
ENTRYPOINT ["uvicorn", "src.main:app"]
CMD ["--host", "0.0.0.0", "--port", "8000"]
```

07 Build Cache Rules

WHY BUILDS ARE SLOW AND HOW TO FIX THEM

1. A changed step invalidates **itself and every step after it**. Order from least to most frequently changing.
2. `COPY` keys on file checksums; `RUN` keys on the command string only. `RUN apt-get update` alone gets cached stale, so chain it with the install.
3. Copy the lockfile, install, then copy source.
4. A `.dockerignore` shrinks the context and stops secrets leaking in:

.DOCKERIGNORE

```
.git
.env*
**/__pycache__
node_modules
.venv
*.ipynb_checkpoints
data/  models/  *.ckpt  *.safetensors
```

5. Cache mounts (`--mount=type=cache`) keep pip, npm, apt and Go caches across builds without putting them in a layer.
6. Secret mounts expose a credential to one `RUN` only, never to a layer or `history`:

SHELL

```
# Dockerfile
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc npm
    ci
# build
docker build --secret id=npmrc,src=$HOME/.npmrc -t web .
```

08 BuildKit and buildx

MULTI-PLATFORM BUILDS AND REMOTE CACHE

BuildKit is the default builder in current Docker Engine. It builds independent stages in parallel, skips stages the target does not need, and supports cache and secret mounts. `buildx` adds multi-platform output and exportable cache, essential for CI runners that start cold.

SHELL

```
docker buildx create --use --name multi
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  --cache-from type=registry,ref=ghcr.io/acme/api:
     buildcache \
  --cache-to   type=registry,ref=ghcr.io/acme/api:
     buildcache,mode=max \
  -t ghcr.io/acme/api:1.4.2 --push .
```

> Building an arm64 image on an amd64 runner uses QEMU emulation and can be 5 to 20 times slower. For heavy builds, use native arm64 runners or cross-compile in the builder stage with `$BUILDPLATFORM`.

09 Choosing a Base Image

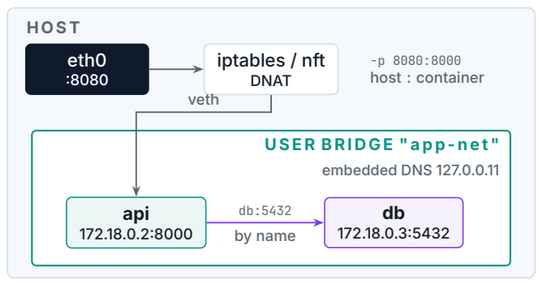
SIZE, COMPATIBILITY, ATTACK SURFACE

Base	Size	Best for, and the catch
`scratch`	0 MB	Static Go or Rust binaries. No shell, no CA certs unless you copy them
`distroless`	2 to 25 MB	Runtime only, glibc, no shell or package manager. Debug via `:debug` tag
`alpine`	~8 MB	Tiny, but musl libc: fewer prebuilt Python wheels, DNS and perf surprises
`*-slim (Debian)`	~75 MB	Safe default for Python and Node: glibc, apt, broad compatibility
`ubuntu`	~78 MB	Matches CUDA and many ML vendor images
`wolfi, chainguard`	small	Minimal, frequently rebuilt, low CVE counts

Sizes are approximate and uncompressed; check with `docker images`.

10 Networking

DRIVERS, DNS, AND PORT PUBLISHING



Driver	Use it when
`bridge`	Default. Single host, NAT to the outside. Create your own for name-based DNS
`host`	Container shares the host network stack. Max throughput, no isolation. Linux only
`none`	No network at all: batch jobs, sandboxed evaluation
`overlay`	Multi-host networks for Swarm
`macvlan, ipvlan`	Container needs its own address on the physical LAN

> Inside a container, `localhost` is the container itself. Reach siblings by service name, and reach the host via `host.docker.internal` (Desktop, or add `--add-host=host.docker.internal:host-gateway` on Linux). The default `bridge` network has *no* name resolution.

11 Storage

VOLUMES, BIND MOUNTS, TMPFS

Type	Flag	Use for
Named volume	`-v pgdata:/data`	Database and model data. Managed by Docker, survives `rm`
Bind mount	`-v ./src:/app/src`	Live source in dev, host config files
tmpfs	`--tmpfs /tmp`	Scratch space and secrets in RAM; pairs with `--read-only`

12 Docker Compose

MULTI-CONTAINER APPS AS CODE

COMPOSE.YAML

```
services:
  api:
    build: .
    ports: ["8000:8000"]
    env_file: .env
    depends_on:
      db: { condition: service_healthy }
    develop:                  # docker compose watch
      watch:
        - { action: sync, path: ./src, target: /app/src }
        - { action: rebuild, path: requirements.txt }
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD_FILE: /run/secrets/db_pw
    secrets: [db_pw]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s
      retries: 10
volumes:
  pgdata:
secrets:
  db_pw: { file: ./secrets/db_pw.txt }
```

Command	Effect
`docker compose up -d --build`	Build if needed, start everything
`docker compose exec db psql -U app`	Run a command in a service
`docker compose down -v`	Stop, remove, *and delete volumes*

Use `compose.yaml` and the `docker compose` plugin; the top-level `version:` key is obsolete. Plain `depends_on` only waits for start, not readiness.

13 Disk Hygiene

WHERE THE GIGABYTES WENT

SHELL

```
docker system df -v                # what uses space
docker builder prune --filter until=72h
docker image prune -a --filter until=168h
docker volume prune -a             # incl. named
docker system prune                # stopped, dangling
```

> **DATA LOSS** Since Engine 23, `docker volume prune` removes only anonymous volumes; `-a` also removes unused *named* volumes. Stop a database stack with `down` and that volume is now "unused".

14 Security Hardening

LEAST PRIVILEGE BY DEFAULT

HARDENED RUN

```
docker run -d --name api \
  --user 10001:10001 \
  --read-only --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --memory 512m --cpus 1.5 --pids-limit 256 \
  --restart unless-stopped \
  -p 127.0.0.1:8000:8000 \
  ghcr.io/acme/api@sha256:<digest>
```

CHECKLIST

- **Non-root:** `USER` in the image; consider rootless Docker or user namespaces on the host.
- **No secrets in images:** not in `ENV`, `ARG` or a `COPY`. Use secret mounts at build and secrets or a vault at runtime.
- **Never** `--privileged`; add back single capabilities with `--cap-add` only when proven necessary.
- **Pin** base images by digest and rebuild on a schedule to pick up patches.
- **Scan** with Docker Scout, Trivy or Grype in CI; fail on critical CVEs with a fix available.
- **SBOM and provenance:** `docker buildx build --sbom=true --provenance=true`.
- **Sign** images (cosign, Notation) and verify at deploy time.

> Ports published with `-p 8000:8000` bind to all interfaces, and Docker writes its own iptables rules, so UFW rules on the host do not block them. Bind to `127.0.0.1` behind a reverse proxy.

15 Runtime Behaviour

PID 1, SIGNALS, LIMITS

- `docker stop` sends `STOPSIGNAL` (SIGTERM), waits 10 s (`-t` to change), then SIGKILL. Handle SIGTERM and drain in-flight requests.
- The kernel gives PID 1 no default signal handlers, and PID 1 must reap zombies. Add `--init` (tini) when the app was not written to be an init process.
- Over `--memory`, the kernel OOM killer fires: exit code **137**, `State.OOMKilled=true`.
- `--cpus` is a CFS quota: throttling shows up as latency, not errors.
- Log to stdout; rotate with `max-size` and `max-file`, or unrotated logs fill disks.

16 GPUs and AI Workloads

SERVING MODELS IN CONTAINERS

Install the NVIDIA Container Toolkit on the host. The image carries CUDA user-space libraries; the host supplies only the driver.

SHELL

```
docker run --rm --gpus all \
  nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
# serve an open model on two GPUs
docker run -d --gpus '"device=0,1"' --ipc=host \
  -p 8000:8000 -e HF_TOKEN \
  -v ~/.cache/huggingface:/root/.cache/huggingface \
  vllm/vllm-openai:latest \
  --model Qwen/Qwen2.5-7B-Instruct \
  --tensor-parallel-size 2
```

- **Weights in volumes, not layers.** Baking 15 GB into an image makes every push, pull and cold start pay for it.
- PyTorch workers use `/dev/shm`; the 64 MB default breaks DataLoaders and NCCL. Use `--ipc=host` or `--shm-size=16g`.
- Choose `cuda:*-runtime` to serve and `cuda:*-devel` only in builder stages that compile kernels.
- In Compose, request GPUs under `deploy.resources.reservations.devices` with `capabilities: [gpu]`.

17 Debugging Playbook

FROM SYMPTOM TO ROOT CAUSE

SHELL

```
docker logs --since 10m api
docker inspect -f '{{.State.OOMKilled}}' api
docker events --filter container=api
# distroless, no shell? borrow a toolbox
docker run --rm -it --pid container:api \
  --network container:api nicolaka/netshoot
```

Exit	Usual cause
`0`	Clean exit. For a server, check that CMD did not exit early
`1`	Application error: read the logs
`125`	The Docker CLI failed: bad flag, name conflict
`126`	Command found but not executable (permissions, CRLF)
`127`	Command not found in PATH or image
`137`	SIGKILL: OOM killer, or stop timeout exceeded
`139`	SIGSEGV: native crash, often a libc or arch mismatch
`143`	SIGTERM: graceful stop

18 Docker to Kubernetes

CONCEPTS THAT CARRY OVER

Docker	Kubernetes
`ENTRYPOINT / CMD`	`command` / `args` (override each)
`-p 8000:8000`	Service, Ingress or Gateway
`-v named:/data`	PersistentVolumeClaim
`--memory, --cpus`	`resources.limits` and `requests`
`HEALTHCHECK`	**Ignored.** Define liveness, readiness, startup probes
`--restart`	Pod `restartPolicy` plus the controller
`compose.yaml`	Deployments, Services, ConfigMaps, Secrets

19 Four Myths

ANSWERS INTERVIEWERS LISTEN FOR

- **"Containers are light VMs."** They are processes on one shared kernel. Use gVisor or Kata for hostile code.
- **"`:latest` is the newest."** It is just the default tag name. Deploy digests.
- **"EXPOSE opens the port."** Only `-p` publishes.
- **"`docker compose down` is safe."** With `-v` it deletes your database volume.